

Epoch 7 Engineering

Competition Onboarding

This onboarding teaches fundamentals of competing in ML competitions through a mock Kaggle-style competition. Each session focuses on one pillar of an ML competition workflow. By the end you will have been acquainted with several important lessons and familiarized yourself with the techstack.

The Mock Competition

We are working with a subset of a dataset focused on identifying bird species from passive acoustic recordings in Africa.

The full dataset covers 264 species; we have filtered it down to the **top 30 most-recorded species on the African subcontinent**, giving us ~4,200 recordings across 30 classes.

Each recording is a variable-length `.ogg` file (median ~30s) captured in the field by citizen scientists. The task is to classify the **primary species** present in a recording. You will train a model and submit predictions to the leaderboard — the evaluation metric is **mAP** (see Session 1 below).

Submitting to the leaderboard

Once you have trained a model, run `submit.py` to generate predictions and send them to the leaderboard server:

```
uv run src/submit.py --weights best_model.pt --name "Your Team Name"
```

Argument Required		Description
<code>--weights</code>	yes	Path to your saved <code>.pt</code> model weights
<code>--name</code>	yes	Your team name — this is how you appear on the leaderboard
<code>--model</code>	no	timm model architecture (default: <code>efficientnet_b1</code>) — only change this if you trained a different backbone
<code>--out</code>	no	Where to save the submission CSV (default: <code>submission.csv</code>)

The script will run inference on the test set, save a `submission.csv`, and immediately post it to the server. The server will print your mAP score and a confirmation message.

There are no submission limits, so submit as often as you like. The leaderboard always shows your **best score** across all submissions, so there is no risk in submitting intermediate or experimental results.

Starter pipeline

To let you focus on the concepts rather than boilerplate, a basic pipeline has already been set up:

File	Purpose
<code>precompute_spectrograms.py</code>	Converts all audio files to log-mel spectrograms and caches them as <code>.pt</code> files — run this once before training
<code>prepare_dataset.py</code>	PyTorch <code>Dataset</code> that loads cached spectrograms and returns <code>(spec, label)</code> pairs
<code>metric.py</code>	mAP implementation using sklearn
<code>train.py</code>	Basic training loop with EfficientNet-B1 from timm

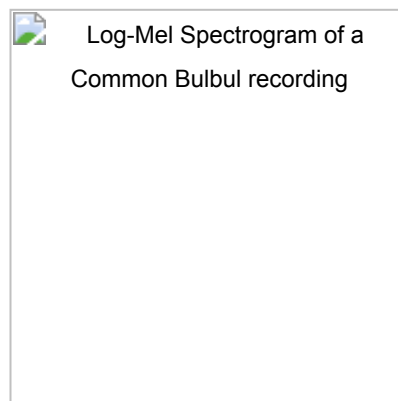
The audio-to-spectrogram conversion is already handled for you. For a bird classification task this is a natural choice: bird calls have distinctive frequency patterns that are clearly visible in the spectrogram, and it lets us treat the problem as standard image classification — plugging in any pretrained CNN backbone off the shelf.

From audio to model input — the Log-Mel Spectrogram

To directly exploit patterns in the frequency domain, we convert audio into an image-like representation: the **log-mel spectrogram**. What happens?

1. Compute the Short-Time Fourier Transform (STFT) to get frequency content over time
2. Map the frequency axis onto the **mel scale** — a perceptual scale that mirrors how ears work, compressing high frequencies
3. Take the **log** of the power, converting to decibels — this compresses the dynamic range and makes the representation more linearly separable

The result is a 2D array (mel bins × time frames) that we treat as a single-channel image and feed into a standard image classifier.



Each vertical slice is one 10ms window of audio. Brighter = more energy at that frequency at that moment. The repeated chirp pattern is the Common Bulbul's call.

Session 1 — Foundations & Validation

Goal: Build the habit of trusting your feedback loop before anything else. A CV score you can rely on is worth more than any model improvement in the first weeks of a competition. If your metric is lying to you, every local improvement is untrustworthy.

Environment setup with `uv`

Why a virtual environment?

Every project needs its own isolated set of dependencies. Without one, installing package A for project 1 can silently break project 2 when A gets upgraded. A virtual environment (`.venv`) is a self-contained Python installation scoped to this project.

Why `uv`?

`uv` is a Python package manager written in Rust. It is a drop-in replacement for `pip` + `venv` that is 10–100× faster and handles everything in one tool: creating environments, resolving dependencies, locking versions, and running scripts.

```
# install a package and add it to pyproject.toml
uv add torch torchaudio timm scikit-learn

# recreate the exact environment from the lockfile (e.g. after cloning the repo)
uv sync

# run a script inside the managed environment
uv run train.py
```

`uv sync` is the command you run after cloning — it reads `uv.lock` and installs the exact pinned versions everyone else on the project is using.

The metric — mean Average Precision (mAP)

In each competition, it is crucial to properly understand the metric, since that is what you are spending your time optimizing.

In this competition the metric is mAP: mAP measures how well your model **ranks** the correct class. For each class, Average Precision (AP) summarises the precision-recall curve. It rewards a model that puts the right answer near the top of its confidence scores, not just above a threshold. mAP is the mean of AP across all 30 classes.

```
from sklearn.metrics import average_precision_score

# targets: (N, 30) binary matrix
# preds:   (N, 30) probability matrix (sigmoid outputs, NOT logits)
map_score = average_precision_score(targets, preds, average="macro")
```

Key properties to understand:

- It is **threshold-free** — you are optimising a ranking, not a binary decision
- It is **macro-averaged** — a rare class counts as much as a common one, so class imbalance hurts you directly
- A model that outputs 0.5 for every class scores near zero; the signal is in relative ordering

Cross-validation and OOF splits

You have a limited number of daily leaderboard submissions, so you cannot rely on the public leaderboard to evaluate every change you make. Instead, you reserve part of your own training data as a **validation set** and evaluate locally — this is your primary feedback signal.

The simplest approach is a single **hold-out split**: keep 80% for training, 20% for validation. It works, but it is noisy — your score depends heavily on which 20% you happened to hold out. A better approach is **k-fold cross-validation**: split the data into k equal folds, train k models each leaving out a different fold, and aggregate the results. A larger k gives a less noisy estimate of generalisation, at the cost of k times the compute.

Not all splits are created equal

The goal of your validation set is to mock the hidden test set as closely as possible. If information leaks from your training set into your validation set, your local score will be artificially inflated — and you will only discover this when your leaderboard score is much lower than expected.

Before writing any split code, look carefully at the metadata: who collected these recordings, where, and how? Ask yourself what the test set likely looks like compared to the training set, and whether a naive random split would respect that boundary. Think about what the atomic unit of your split should be — is it a single 5-second clip, a full recording, or something else?

Getting this right is the most important engineering decision in Session 1. A fast model with a trustworthy CV beats a slow model you cannot evaluate.

Save your OOF predictions

With k-fold CV, each model predicts on its held-out fold — the samples it never trained on. Concatenating these predictions across all folds gives you one prediction per training example, collectively called **out-of-fold (OOF) predictions**. Save them to disk after every experiment.

There are three reasons this matters:

1. **A single honest score.** Computing mAP on the full OOF array is more reliable than averaging the per-fold scores, because it weights each sample equally rather than each fold equally.
 2. **A paper trail.** As you iterate, saved OOF files let you go back and understand what earlier models got right or wrong — even after you have stopped tracking that experiment.
 3. **Free ensembling.** When you want to blend multiple models later, you already have their predictions on the full training set ready to combine without rerunning anything.
-